

LANDASAN TEORI & PERANCANGAN SISTEM

Fitur Rekomendasi Layanan dengan Metode *Transaction-Based Recommendation*

*Perancangan Website Barbershop dengan Fitur Rekomendasi Layanan
Menggunakan Metode Transaction-Based Recommendation*

Dokumen ini disusun dengan gaya penulisan akademik dan dapat digunakan sebagai bahan **Bab II (Landasan Teori)** dan **Bab III (Analisis & Perancangan Sistem)** pada penelitian ilmiah.

2.1 Pengertian *Transaction-Based Recommendation*

Transaction-Based Recommendation (rekomendasi berbasis transaksi) adalah salah satu pendekatan dalam sistem rekomendasi (*recommender system*) yang menghasilkan saran item kepada pengguna dengan menganalisis **riwayat data transaksi** yang tersimpan di dalam basis data. Berbeda dengan pendekatan *content-based filtering* yang menilai kemiripan atribut item, atau *collaborative filtering* yang membutuhkan matriks preferensi antar-pengguna, metode berbasis transaksi memanfaatkan **frekuensi kemunculan** suatu item pada kumpulan transaksi sebagai indikator utama tingkat popularitas dan relevansinya.

Secara konseptual, metode ini berlandaskan asumsi bahwa **item yang paling sering ditransaksikan oleh keseluruhan pengguna merupakan item yang paling diminati**, sehingga layak direkomendasikan kepada pengguna lain. Pendekatan ini termasuk dalam kategori *popularity-based recommendation* yang memanfaatkan data perilaku kolektif (*collective behavior*) sebagai dasar pengambilan keputusan rekomendasi.

Dalam konteks penelitian ini, satu **transaksi** didefinisikan sebagai satu catatan pemesanan (*booking*) layanan oleh pelanggan yang tersimpan pada entitas `bookings`. Setiap transaksi merujuk pada satu layanan tertentu (`service_id`), sehingga akumulasi jumlah transaksi per layanan dapat dijadikan ukuran kuantitatif popularitas layanan tersebut.

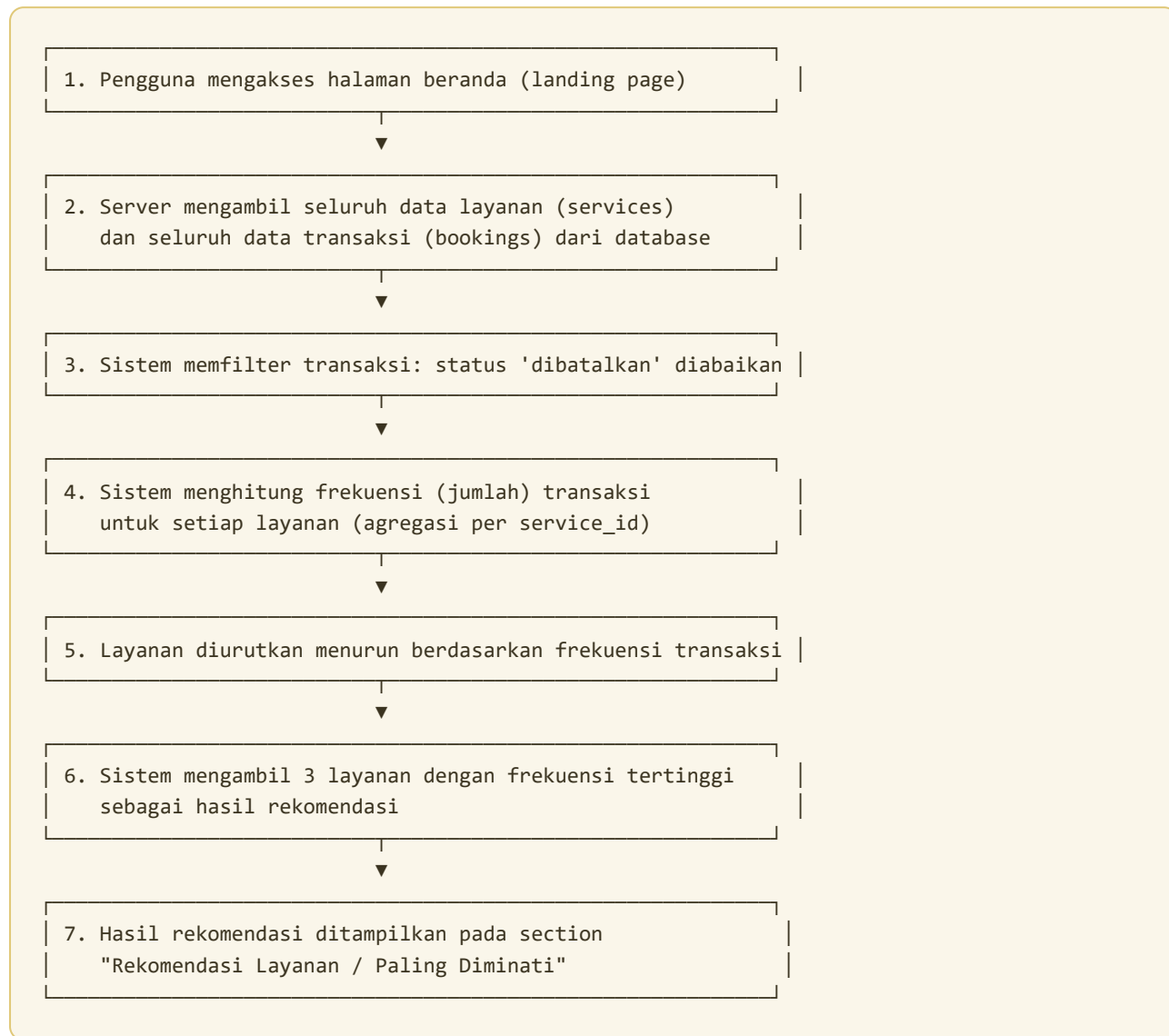
2.2 Alasan Metode Ini Cocok untuk Website Barbershop

Pemilihan metode *transaction-based recommendation* pada perancangan website barbershop didasarkan pada beberapa pertimbangan berikut:

1. **Karakteristik data transaksi yang sederhana dan terstruktur.** Data pemesanan barbershop bersifat eksplisit dan mudah dikuantifikasi (pelanggan, layanan, tanggal, jam). Hal ini sangat sesuai dengan pendekatan berbasis frekuensi transaksi yang tidak memerlukan data preferensi kompleks.
2. **Mengatasi masalah *cold-start* pada pengguna baru.** Metode *collaborative filtering* memerlukan riwayat interaksi yang cukup banyak dari setiap pengguna. Pada barbershop, mayoritas pengunjung adalah calon pelanggan baru yang belum memiliki riwayat. Rekomendasi berbasis transaksi tetap mampu memberikan saran relevan (layanan terpopuler) tanpa bergantung pada riwayat individu.
3. **Jumlah item (layanan) relatif sedikit dan stabil.** Barbershop umumnya hanya menawarkan beberapa hingga belasan jenis layanan. Dengan ruang item yang kecil, perhitungan berbasis frekuensi menjadi sangat efisien dan mudah diinterpretasikan.
4. **Komputasi ringan dan dapat berjalan secara *real-time*.** Perhitungan agregasi frekuensi memiliki kompleksitas rendah sehingga dapat dieksekusi langsung saat halaman dimuat (*server-side rendering*) tanpa proses pelatihan model (*training*) yang berat.
5. **Relevansi bisnis yang tinggi.** Menampilkan layanan terpopuler mendorong calon pelanggan memilih layanan yang terbukti diminati, sekaligus membantu pemilik mengidentifikasi tren permintaan pasar.

3.1 Alur Kerja Sistem Rekomendasi

Alur kerja sistem rekomendasi pada website FI Barbershop dapat diuraikan dalam tahapan berikut:



Keseluruhan proses dieksekusi di sisi server (*Server Component*) sehingga hasil rekomendasi sudah terbentuk sebelum halaman dikirimkan ke peramban (*browser*) pengguna.

3.2 Data yang Digunakan

Sistem rekomendasi memanfaatkan dua entitas utama dari basis data PostgreSQL (Supabase):

a. Entitas `services` (Layanan)

Atribut	Tipe	Keterangan
<code>id</code>	UUID	Pengenal unik layanan
<code>name</code>	Teks	Nama layanan
<code>price</code>	Integer	Harga layanan
<code>duration</code>	Integer	Durasi layanan (menit)

b. Entitas `bookings` (Transaksi)

Atribut	Tipe	Keterangan
<code>id</code>	UUID	Pengenal unik transaksi
<code>customer_id</code>	UUID	Pelanggan yang memesan
<code>service_id</code>	UUID	Layanan yang dipesan (kunci agregasi)
<code>status</code>	Teks	menunggu, dikonfirmasi, selesai, atau dibatalkan

Atribut kunci dalam perhitungan adalah `service_id` pada entitas `bookings`, yang digunakan untuk menghitung frekuensi transaksi tiap layanan. Atribut `status` digunakan sebagai **filter validitas transaksi** — transaksi dengan status `dibatalkan` tidak diperhitungkan agar popularitas mencerminkan permintaan nyata.

3.3 Proses Perhitungan Rekomendasi

Notasi dan Formula

Misalkan:

- $S = \{s_1, s_2, \dots, s_n\}$ adalah himpunan seluruh layanan.
- $B = \{b_1, b_2, \dots, b_m\}$ adalah himpunan seluruh transaksi *valid* (status $\neq$ dibatalkan).
- Setiap transaksi b_j memiliki atribut layanan $service(b_j)$.

Skor popularitas (frekuensi transaksi) untuk setiap layanan s_i dihitung sebagai:

$$f(s_i) = \sum_{j=1..m} \mathbb{1}[service(b_j) = s_i]$$

di mana $\mathbb{1}[\cdot]$ adalah fungsi indikator yang bernilai 1 jika kondisi benar dan 0 jika salah. Selanjutnya seluruh layanan diurutkan secara **menurun (*descending*)** berdasarkan nilai $f(s_i)$, dan himpunan rekomendasi akhir adalah k layanan teratas (pada implementasi ini $k = 3$):

$$R_{\text{top}} = \{r_1, r_2, r_3\} \subseteq \text{sort}_{\downarrow}(S, f(s_i))$$

Implementasi Algoritma (Pseudocode)

```

INPUT  : daftar layanan S, daftar transaksi B
OUTPUT : 3 layanan rekomendasi teratas

1. counts ← peta kosong { service_id → jumlah }
2. UNTUK setiap transaksi b DALAM B:
3.     JIKA b.status ≠ 'dibatalkan':
4.         counts[b.service_id] ← counts[b.service_id] + 1
5. UNTUK setiap layanan s DALAM S:
6.     s.count ← counts[s.id] (0 jika tidak ada)
7. urutkan S secara menurun berdasarkan s.count
8. KEMBALIKAN 3 layanan pertama dari S

```

Realisasi pada Kode Program

Algoritma di atas diimplementasikan pada berkas `components/landing/recommend-data.tsx` :

```
async function getRecs(): Promise<ServiceWithCount[]> {
  const supabase = await createClient()
  const { data: services } = await supabase.from('services').select('*')
  const { data: bookings } = await supabase
    .from('bookings')
    .select('service_id')
    .neq('status', 'dibatalkan') // filter transaksi valid

  // Agregasi frekuensi transaksi per layanan
  const counts: Record<string, number> = {}
  bookings?.forEach((b) => {
    counts[b.service_id] = (counts[b.service_id] || 0) + 1
  })

  // Pemberian skor, pengurutan menurun, ambil 3 teratas
  return (services || []).
    .map((s) => ({ ...s, count: counts[s.id] || 0 }))
    .sort((a, b) => b.count - a.count)
    .slice(0, 3)
}
```

3.4 Contoh Studi Kasus Sederhana

Diasumsikan sebuah barbershop memiliki **5 layanan** dan telah mencatat sejumlah transaksi sebagaimana tabel berikut.

Tabel data transaksi (bookings):

No	Layanan Dipesan	Status
1	Potongan Rambut Klasik	selesai
2	Potongan Rambut Klasik	dikonfirmasi
3	Skin Fade	selesai
4	Potongan Rambut Klasik	menunggu
5	Cuci & Tata Rambut	selesai
6	Skin Fade	dibatalkan
7	Cuci & Tata Rambut	selesai
8	Mewarnai Rambut	menunggu

Langkah 1 — Filter transaksi valid (buang status **dibatalkan**): Transaksi No. 6 (Skin Fade) diabaikan.

Langkah 2 — Hitung frekuensi per layanan $f(s_i)$:

Layanan	Perhitungan	Frekuensi $f(s_i)$
Potongan Rambut Klasik	No. 1, 2, 4	3
Cuci & Tata Rambut	No. 5, 7	2
Skin Fade	No. 3 (No. 6 dibuang)	1
Mewarnai Rambut	No. 8	1
Perawatan Jenggot	—	0

Langkah 3 — Urutkan menurun & ambil 3 teratas:

Peringkat	Layanan	Frekuensi
#1 🏆	Potongan Rambut Klasik	3
#2 🥈	Cuci & Tata Rambut	2
#3 🥉	Skin Fade	1

Hasil rekomendasi yang ditampilkan kepada pengunjung adalah: **Potongan Rambut Klasik, Cuci & Tata Rambut**, dan **Skin Fade**.

Studi kasus ini menunjukkan bahwa metode berhasil mengangkat layanan yang paling sering ditransaksikan, sekaligus membuktikan bahwa transaksi yang dibatalkan tidak memengaruhi hasil rekomendasi (Skin Fade tetap berperingkat berdasarkan satu transaksi valid, bukan dua).

3.5 Manfaat Fitur Rekomendasi

a. Bagi Pelanggan (*Customer*)

1. **Mempermudah pengambilan keputusan** — calon pelanggan, khususnya pengguna baru, memperoleh panduan layanan mana yang paling diminati tanpa perlu menelusuri seluruh daftar.
2. **Meningkatkan rasa percaya (*social proof*)** — rekomendasi dari data transaksi nyata memberikan keyakinan bahwa layanan tersebut telah terbukti diminati pelanggan lain.
3. **Menghemat waktu** — proses pemilihan layanan menjadi lebih cepat dan efisien.

b. Bagi Pemilik Barbershop (*Owner / Admin*)

1. **Identifikasi tren permintaan** — pemilik dapat mengetahui layanan unggulan untuk dijadikan fokus promosi atau penambahan kapasitas.
2. **Dasar pengambilan keputusan bisnis** — data popularitas dapat digunakan untuk strategi penetapan harga, paket *bundling*, maupun evaluasi layanan yang kurang diminati.
3. **Peningkatan konversi pemesanan** — penonjolan layanan populer berpotensi meningkatkan jumlah transaksi karena pelanggan terarah pada pilihan yang relevan.
4. **Otomatisasi tanpa intervensi manual** — rekomendasi diperbarui otomatis seiring bertambahnya transaksi, tanpa pengaturan manual oleh admin.

3.6 Kesimpulan

Metode *Transaction-Based Recommendation* terbukti sesuai untuk diterapkan pada website barbershop karena karakteristik datanya yang sederhana, kemampuannya mengatasi masalah *cold-start*, efisiensi komputasi yang tinggi, serta relevansi bisnis yang langsung dapat dirasakan. Melalui agregasi frekuensi transaksi pada entitas `bookings`, sistem mampu menghasilkan rekomendasi tiga layanan terpopuler secara otomatis dan *real-time*, yang bermanfaat baik bagi pelanggan dalam pengambilan keputusan maupun bagi pemilik dalam analisis bisnis.